

ENIGMA: эволюция когнитивной архитектуры (по истории репозитория)

0. Рамка анализа (что именно сравнивалось)

Анализ выполнен по артефактам репозитория, без психологических интерпретаций:

- Git-история и срезы по коммитам:
 - a78c3e2 (2026-03-06), 968b505 (2026-03-06) — ранний main
 - e4ddc7b (2026-05-10), 63a3084 (2026-05-11) — CFRM-рывок
 - 95c7480 (2026-05-12), ec669d1 (2026-05-14) — ранние sandbox ветки
 - 7818e88 (2026-05-14) — закрепление governance/enforcement
- Системные документы:
 - docs/Tasks/ADR (Architecture Decision Records).md (ADR-001..051)
 - docs/Tasks/АРХИТЕКТУРНЫЙ_УСТАВ_ENIGMA.md
 - docs/Tasks/ADR-000_IMPACT_TEMPLATE.md
 - docs/Tasks/Правила Фикса БАГОВ.md
 - docs/audits/ADR-048_IMPACT.md ... ADR-051_IMPACT.md
 - docs/INFO/Fase + ADM.md, docs/INFO/ENIGMA_global_table.md
- Эволюционные отчёты:
 - COMPARISON_REPORT_V.0.5.3.0.1_НОВАЯ_РЕАЛЬНОСТЬ_1.md
 - COMPARISON_REPORT_V.0.5.3.0.2_НОВАЯ_РЕАЛЬНОСТЬ_2.md
 - COMPARISON_REPORT_V.0.5.3.0.4_ПЕСОЧНИЦЫ_1_vs_V.0.5.3.0.3_НОВАЯ_РЕАЛЬНОСТЬ_3.md
 - COMPARISON_REPORT_V.0.5.3.0.5_ПЕСОЧНИЦЫ_2.md
 - COMPARISON_REPORT_V.0.5.3.0.6_ПЕСОЧНИЦЫ_3.md

1. Реконструкция стиля принятия решений (по эпохам)

А. Ранний main (06.03.2026 -> 15.03.2026)

Материал эпохи:

- main в состоянии 968b505 (README, backend/app/services/orchestrator.py)
- baseline d17cc33 как ранний operational snapshot

Стиль решений:

- Приоритет на **рабочий end-to-end контур** (MVP-ориентация): быстро собрать работающий цикл, затем расширять.
- Архитектурная декомпозиция уже есть (агенты, memory layers), но контроль целостности в основном **через структурирование модулей**, не через формальные архитектурные

запреты.

- Управляющая идея: Python считает, LLM озвучивает — это ранний прототип будущей причинной доктрины.

Тип выбора в спорных местах:

- Выбор в пользу **производственной выполнимости** (параллелизм агентных вызовов, system requirements guard, JSONL как стартовое хранилище).
- Компромисс: допускается более высокий архитектурный долг ради быстрого функционального покрытия.

B. Ранние sandbox branches (V.0.5.3.0.4/0.5, 12.05.2026 -> 14.05.2026)

Материал эпохи:

- 95c7480, ec669d1
- sandbox-вертикали (micro, oscilloscope, phenomenology)
- ADR-000_IMPACT_TEMPLATE.md

Стиль решений:

- Переход от «добавить механику» к «**доказать причинную корректность механики**».
- Появляется инженерный ритуал: change -> causal sandbox -> impact-formalization.
- Решения больше не закрываются фактом «работает», а закрываются фактом «**замкнут контур и есть тест наблюдаемости**».

Тип выбора в спорных местах:

- Приоритет у проверяемости (closed-loop тесты, compliance-контуры), даже если это замедляет feature throughput.

C. CFRM-эпоха (V.0.5.3.0.1/0.2, 10.05.2026 -> 11.05.2026)

Материал эпохи:

- e4ddc7b, 63a3084
- ADR-024..033 (особенно ADR-025, 029, 033)

Стиль решений:

- Архитектор меняет не поведение модулей, а **онтологию runtime**:
 - от глобального world-state к локальным причинным пузырям,
 - от событий как «фактов истины» к FieldDisturbance и проекционной феноменологии.
- Принятие решений идет через схему: контекст причинного дефекта -> новая модель данных -> bridge для legacy -> пошаговая миграция.

Тип выбора в спорных местах:

- Выбор в пользу **масштабируемой причинной модели** даже при риске краткосрочной сложности миграции.
- Активно используются «переходные конструкции» (Legacy Bridge, Controlled Degradation Adapter), чтобы не останавливать поставку.

D. Governance/Enforcement-эпоха (12.05.2026 -> 14.05.2026)

Материал эпохи:

- ADR-043..051
- docs/Tasks/АРХИТЕКТУРНЫЙ_УСТАВ_ENIGMA.md
- docs/Tasks/Правила Фикса БАГОВ.md
- docs/audits/ADR-048_IMPACT.md ... ADR-051_IMPACT.md

Стиль решений:

- Решения переводятся в **режим конституции**:
 - «закон/запрет/обязательная процедура»,
 - фиксированные слои ответственности,
 - rollback-процедуры как часть определения изменения.
- Архитектурное управление становится двухконтурным:
 - контур исполнения (tick/runtime),
 - контур принуждения (устав, impact audit, bug protocol).

Тип выбора в спорных местах:

- Приоритет у **дисциплины эволюции**: допускаются только изменения, у которых есть downstream-карта, impact-оценка и sandbox-подтверждение.

2. Механизмы борьбы с complexity

2.1. Фазовая декомпозиция и темпоральная дисциплина

- Эволюция от общего orchestrator-контейнера к 10-фазной модели (TickOrchestrator, Устав §3).
- Снятие «скрытых мутаций» через ограничение места, где изменения вообще имеют право происходить.

2.2. Контрактная типизация и доменная алгебра

- Переход к EventDT0, StateDeltas v2, доменным payload, DecisionContext.
- Complexity разрезается не по файлам, а по **семантическим границам доменов** (social/emotion/physiology/perception).

2.3. Локализация причинности (CFRM)

- Удаляется единый глобальный центр редукции.
- Complexity превращается из $O(\text{global state reconciliation})$ в локальные causal neighborhoods.

2.4. Controlled Degradation для миграций

- Новые модели внедряются с адаптерами к legacy (v2 -> adapter -> v1), а не «Big Bang rewrite».
- Это снижает риск остановки системы при смене онтологии.

2.5. Верификация поведения вместо верификации строк

- tests/sandbox/* фокусируются на causal laws (closure, erosion, divergence), а не на поверхностных asserts состояния.

2.6. Governance как anti-entropy механизм

- Устав + Impact Audit + Rollback + Bug Protocol формируют «архитектурный иммунитет» против деградации границ.

3. Recurring abstractions (повторяющиеся абстракции)

1. Orchestrator/Phase control plane

От GameOrchestrator раннего main к TickOrchestrator с явной фазовой конституцией.

2. Intent -> Pressure -> Decision цепь

Повторяется в разных формах, но становится всё более формальной:

- ранний intent path,
- затем Will/Pressure,
- затем DecisionContext geometry.

3. Единый язык переносимых фактов

EventDT0, typed payload, domain tags, registry контрактов.

4. Single source of truth паттерн

Сначала для данных памяти/сервиса, затем жёстко для пространства (ADR-048), затем для каузальной проекции.

5. Bridge/Adapter pattern как стратегический инструмент

Legacy bridge, degradation adapters, boundary adapters — постоянный способ удержания эволюции без полного обвала обратной совместимости.

6. Наблюдательность причинности

CausalTrace, sandbox probes, impact-аудиты — повторяющийся контур «сначала измерить

причинную корректность, потом считать изменение принятым».

4. Какие проблемы считаются фундаментальными

По структуре ADR и enforcement-документов фундаментальными считаются не «фичевые», а системные проблемы:

1. Разрыв причинной цепи

Если команда/событие не доходит до решения через формальную топологию, система считается сломанной.

2. Множественные конкурирующие истины

Особенно spatial truth (ADR-048): любое дублирование authority трактуется как первичный риск.

3. Скрытая мутация и ownership ambiguity

Главная цель — однозначный владелец стадии и запрет неявных side-effects.

4. Непроверяемая эмерджентность

Эмерджентность допустима только если ее можно наблюдать детерминированно в sandbox.

5. Темпоральная фальсификация

Retro-simulation без наблюдаемого пайплайна (ADR-047) трактуется как фундаментальная онтологическая ошибка.

6. Дрейф контрактов при быстрых изменениях

Отсюда жёсткие протоколы фикса багов, шаблоны impact-аудита и rollback-обязательства.

5. Как меняется уровень системного мышления со временем

Уровень 1: Компонентный (ранний main)

- Основная единица мышления: сервис/агент.
- Критерий успеха: работоспособный pipeline и функциональный охват.

Уровень 2: Пайплайновый (майские рефакторы до CFRM)

- Единица мышления: стадия обработки + порядок фаз.
- Критерий успеха: отсутствие рассыпания между соседними слоями.

Уровень 3: Онтологический (CFRM эпоха)

- Единица мышления: модель реальности (что такое событие/восприятие/давление).
- Критерий успеха: причинная согласованность модели при масштабировании.

Уровень 4: Метасистемный (governance/enforcement)

- Единица мышления: правила изменения правил.
- Критерий успеха: система способна эволюционировать без потери архитектурной идентичности.

Итог эволюции:

Траектория сместилась от «собрать работающую систему» к «управлять эволюцией причинной системы через формальные механизмы принуждения».

6. Сравнение эпох в одном кадре

Эпоха	Единица проектирования	Как режется сложность	Что считается доказательством корректности
Ранний main	Сервис/агент	Модульная декомпозиция + параллелизм	Рабочий сценарий + базовые тесты
Ранние sandbox ветки	Каузальный контур	Sandbox-вертикали + probes	Closed-loop прохождение причинной цепи
CFRM эпоха	Онтология причинности	Локальные пузыри + projection/reduction	Согласованность модели и runtime-интеграции
Governance/enforcement	Арх. конституция	Устав + impact + rollback + баг-протокол	Выполнение процедур + отсутствие контрактного дрейфа

7. Финальная структура вывода (по формату запроса)

1. Критический параметр решения

Ключевой параметр всей эволюции: **степень замкнутости и проверяемости причинного контракта** (Intent -> Pressure -> Decision -> State -> Snapshot) при каждом архитектурном изменении.

2. Возможные сценарии при минимальном смещении входных данных

1. Ослабление enforcement-процедур при той же скорости изменений -> быстрый возврат hidden divergence между слоями.
2. Сохранение sandbox-first + impact-аудитов -> стабильный рост сложности без потери наблюдаемости.

3. Усиление онтологии без поддержки adapters -> локальные ускорения, но рост migration-friction и риск регрессий downstream.

3. Что может перевернуть итоговый вывод

1. Массовые переписывания без ADR/impact/rollback трассировки.
2. Переход к non-deterministic валидации вместо causal sandbox.
3. Восстановление дублирующих authority-слоёв (особенно spatial/time).

4. Альтернативные варианты с вероятностной оценкой (0-100%)

1. **Продолжение текущей линии (ontology + enforcement + sandbox): 82%**
Наиболее вероятно сохранит управляемую эволюцию когнитивной архитектуры.
2. **Смещение в сторону feature-first без жёсткого governance: 41%**
Даст краткосрочный throughput, но с высокой вероятностью дрейфа контрактов.
3. **Жёсткая заморозка онтологии и фокус только на presentation: 28%**
Снижает риски миграции, но останавливает развитие причинного ядра.

5. Краткий, но глубокий вывод

Эволюция ENIGMA показывает переход от компонентного проектирования к управлению онтологией причинности и далее к метасистеме архитектурного принуждения.

Главный сдвиг не в усложнении кода, а в появлении механизма, который контролирует, **как именно** системе разрешено усложняться.